Digital System Design Report 3

Group 29

Authors: Qichao (Arlo) Wang (qw823)

1 Scope and Objectives

This report covers Task 7 (7a/7b/7c) and Task 8 of the coursework. The target function is

$$F(x) = \sum_i f(x_i), \quad f(x) = 0.5x + x^3 \cos\left(\frac{x-128}{128}\right)$$

under single-precision floating-point representation.

2 Experimental Methodology

2.1 Hardware/Software Setup

To fill:

- FPGA board and clock target (50 MHz).
- Quartus / Nios II EDS / simulator versions.
- Compiler flags and BSP settings.

2.2 Test Cases and Metrics

Use the three coursework test vectors:

- C1: $X = 0 : 5 : 255$
- C2: $X = 0 : 1/8 : 255$
- C3: $X = 0 : 1/256 : 255$

Metrics to report:

- Latency / execution time.
- FPGA resources (ALMs/LEs, DSPs, memory bits).
- Numerical error vs double-precision reference.
- Timing closure (worst slack at 50 MHz).

3 Task 7a: CORDIC Analysis (10%)

3.1 Reference Model and Statistical Method

Task 7a uses a MATLAB Monte Carlo model with:

- random input $\theta \sim U[-1, 1]$, quantised to `single`;
- reference output $y_{ref} = \cos(\theta)$ under MATLAB single precision;
- CORDIC estimate y_{cordic} using fixed-point internal arithmetic;
- error sample $e_k = (y_{cordic,k} - y_{ref,k})^2$;
- MSE estimate $\hat{\mu}_e = \frac{1}{N}\sum_{k=1}^{N} e_k$ and 95% CI $\hat{\mu}_e \pm 1.96 \cdot s_e/\sqrt{N}$.

For all reported Task 7a results, we used $N = 50{,}000$ samples and seed $= 7$.

3.2 Design Space Exploration

We swept both CORDIC parameters required by the specification:

- iterations: all integers $N_{iter} \in \{12, 13, \ldots, 24\}$;
- fixed-point precision: all integer FRAC values from 20 to 34.

The total fixed-point width is set as:

$$W = \text{FRAC} + 2.$$

Table 1: Task 7a sweep excerpt around the pass boundary.

N_iter	W	FRAC	2^{-i} est.	MSE	95% CI upper	Pass
16	22	20	1.526e-05	8.817e-11	8.926e-11	No
17	22	20	7.629e-06	2.575e-11	2.607e-11	No
18	22	20	3.815e-06	1.000e-11	1.013e-11	Yes
16	23	21	1.526e-05	8.527e-11	8.633e-11	No
17	23	21	7.629e-06	2.254e-11	2.282e-11	Yes
18	23	21	3.815e-06	6.555e-12	6.640e-12	Yes
16	24	22	1.526e-05	8.444e-11	8.548e-11	No
17	24	22	7.629e-06	2.163e-11	2.190e-11	Yes
18	24	22	3.815e-06	5.599e-12	5.668e-12	Yes

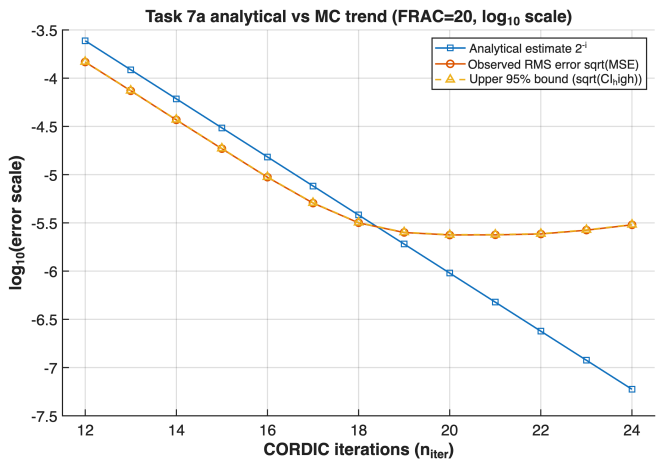

Figure 1: Analytical 2^{-i} trend versus observed Monte Carlo error scale (shown in $\log_{10}$ domain for readability).

The +2 integer/sign bits cover the CORDIC dynamic range for $\theta \in [-1, 1]$ and $\cos(\theta) \in [-1, 1]$. The acceptance criterion is the coursework requirement:

$$\text{CI}_{95,\text{upper}}(\text{MSE}) < 2.4 \times 10^{-11}.$$

3.3 Analytical Error Estimate (2^{-i})

For rotation-mode CORDIC, the residual angle after i iterations is bounded by a term proportional to 2^{-i}. Using $|\Delta\cos(\theta)| \approx |\sin(\theta)\Delta\theta| \le |\Delta\theta|$, the absolute cosine error also follows an $O(2^{-i})$ trend; therefore squared error follows approximately $O(2^{-2i})$. This estimate is not used as the final proof, but it is effective to prune the search space before Monte Carlo.

3.4 Selected Configuration and Justification

After including odd values, the first passing point in lexicographic order (minimum iterations, then minimum FRAC) is:

$$(N_{iter}, W, \text{FRAC}) = (17, 23, 21).$$

Measured statistics for this minimum-passing point are:

- MSE $= 2.254 \times 10^{-11}$,
- 95% CI $= [2.226, 2.282] \times 10^{-11}$,
- CI upper bound margin to spec: $2.4 \times 10^{-11}/2.282 \times 10^{-11} \approx 1.05\times$.

For implementation in Task 7b/7c we keep $(18, 24, 22)$ as a conservative choice: with `ITER_PER_CYCLE`=3, both 17

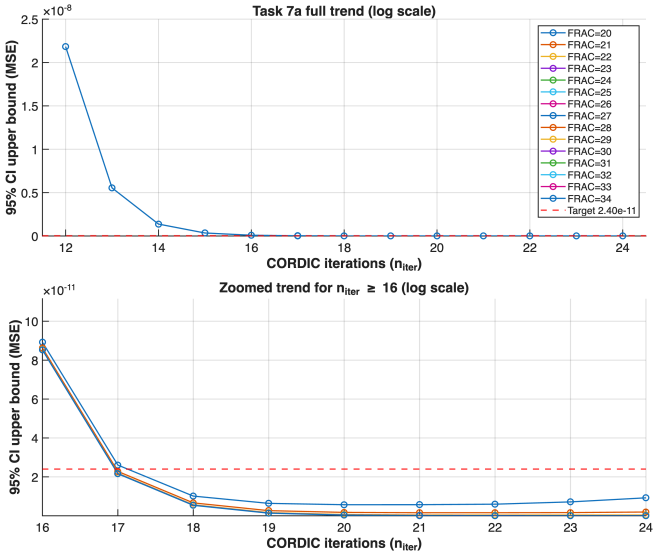

Figure 2: 95% CI upper bound of MSE versus iterations for each fixed-point precision (top: full sweep; bottom: zoom for $N_{iter} \geq 16$, both on log scale).

and 18 iterations map to the same CORDIC cycle count (6 cycles), while $(18, 24, 22)$ has significantly larger numerical margin.

3.5 Impact on Accuracy and Latency

Observed impact from the full sweep:

- For FRAC=20, the threshold is crossed at $N_{iter} = 18$ (17 fails, 18 passes).
- For FRAC≥21, $N_{iter} = 17$ already passes.
- Increasing FRAC beyond the threshold region keeps reducing error, but with diminishing practical benefit.
- Latency grows with iteration count (for fixed iterations-per-cycle), while larger FRAC increases datapath cost; therefore minimum-passing and implementation-conservative points are both reported.

4 Task 7b: CORDIC Architecture and Implementation (10%)

4.1 CORDIC Microarchitecture

To fill:

- Rotation mode equations and pipeline/stage arrangement.
- Number of HW stages and iterations per cycle.
- Why LUT or vendor cosine IP is not used.
- Comparison of single-iteration/cycle vs multi-iteration/cycle options.

Placeholder: CORDIC datapath/FSM diagram

Figure 3: CORDIC architecture used in Task 7b.

4.2 Verification and Timing

To fill:

Table 2: Step-2 per-accelerator latency (placeholder).

Block	Predicted cycles	Measured cycles
FP_MUL	TODO	TODO
FP_ADD/SUB	TODO	TODO
COS (CORDIC)	TODO	TODO

Table 3: Step-2 whole-function timing under Nios scheduling (placeholder).

Case	Total time (ms)	Sum accel time (ms)	Overhead (ms)
C1	TODO	TODO	TODO
C2	TODO	TODO	TODO
C3	TODO	TODO	TODO

- Simulation waveform checks (input/output and done timing).
- Measured latency in cycles.
- Timing report summary at 50 MHz.
- If timing fails for any candidate architecture, document it and explain.

5 Task 7c: Single Custom Instruction Integration (15%)

5.1 Step 2: Three-Accelerator System

Describe the intermediate architecture with separate accelerators: FP_MUL, FP_ADD/SUB, and COS.

5.2 Step 3: Final Single-CI Block

Describe integrated FSM + register file architecture that computes one $f(x)$ call.

5.3 Control and Data Flow

To fill:

- For both Step-2 and Step-3: how data is passed between CPU/custom block/sub-blocks.
- FSM states and transitions.
- Data path registers and intermediate values.
- Resource reuse (single instantiation of each sub-block).

5.4 Performance, Resource, and Error Results

To fill: resource table (ALMs/DSP/M20K) and comparison against Step-2 architecture.

5.5 Expected vs Observed Latency

Discuss discrepancy between isolated block latency and integrated Nios system latency (processor call overhead, memory traffic, blocking CI behavior).

5.6 Standalone vs Nios-Integrated Throughput

To fill:

- Report best-call interval when the block is used standalone (new call immediately after done).
- Report effective call interval when invoked through Nios.
- Explain discrepancy using interface/control overhead.

5.7 Ideal Lower Bound (Dummy Combinational CI)

To fill:

- Implement/measure an ideal one-cycle dummy CI for

Placeholder: final Task 7c block diagram

Figure 4: Task 7c final architecture (single custom instruction for $f(x)$).

Table 4: Task 7c summary across C1/C2/C3 (placeholder).

Metric	C1	C2	C3
Latency of $F(x)$ (ms)	TODO	TODO	TODO
Error vs double reference	TODO	TODO	TODO

C3.

- Report ideal execution-time lower bound and compare with measured results.

6 Task 8: Throughput-Oriented Final Design (10%)

6.1 Architecture Evolution from Task 7

To fill:

- What was changed relative to Task 7 (e.g., accumulation strategy, interface, scheduling).
- Whether CORDIC was reused or redesigned, and why.
- Why the chosen interface level was selected against alternatives.

6.2 Interface Options vs Marking Levels

6.3 Final Task-8 Datapath and Scheduling

6.4 Task 8 Results

6.5 Pipeline Utilisation and Throughput

To fill:

- How often a new x can be accepted (initiation interval).
- Theoretical throughput vs measured throughput.
- Assumptions and bottlenecks (CPU blocking, memory subsystem, CI protocol).

7 Cross-Task Comparison and Discussion

7.1 Performance vs Resource Plot

7.2 Key Trade-offs

- Latency-oriented vs throughput-oriented design decisions.
- CPU orchestration overhead vs internal FSM orchestration.
- Effect of CORDIC precision choice on area and timing.

8 Conclusions

To fill with 4–6 concise bullets:

1. Main achievement for Task 7a/7b/7c.
2. Main achievement for Task 8.
3. Best measured speedup and cost in FPGA resources.
4. Remaining bottlenecks and future improvement opportunities.

Table 5: Task-8 interface option mapping (placeholder).

Option	Description / achieved in this work
L1	Separate CI for $f(x)$ and sum (TODO)
L2	Input $(x, \text{current_sum})$ output new sum (TODO)
L3	Input (x_1, x_2) output $f(x_1)+f(x_2)$ (TODO)
L4	Input (x_1, x_2, op) output with accumulated sum (TODO)
L5	Pipelined: accepts new input before previous inner-function completion (TODO)

Placeholder: Task 8 datapath and control

Figure 5: Task 8 architecture and pipeline schedule.

Appendix A: Reproducibility Checklist

- Git commit hash: TODO
- Toolchain versions: TODO
- Simulation/test commands: TODO
- Quartus report files used: TODO

Table 6: Task 8 performance/resources/error (placeholder).

Metric	C1	C2	C3
Latency of $F(x)$ (ms)	TODO	TODO	TODO
Code size	TODO	TODO	TODO
Error vs double reference	TODO	TODO	TODO

Placeholder: performance vs resources

Figure 6: Performance-resource trade-off from baseline to Task 8 final design.